

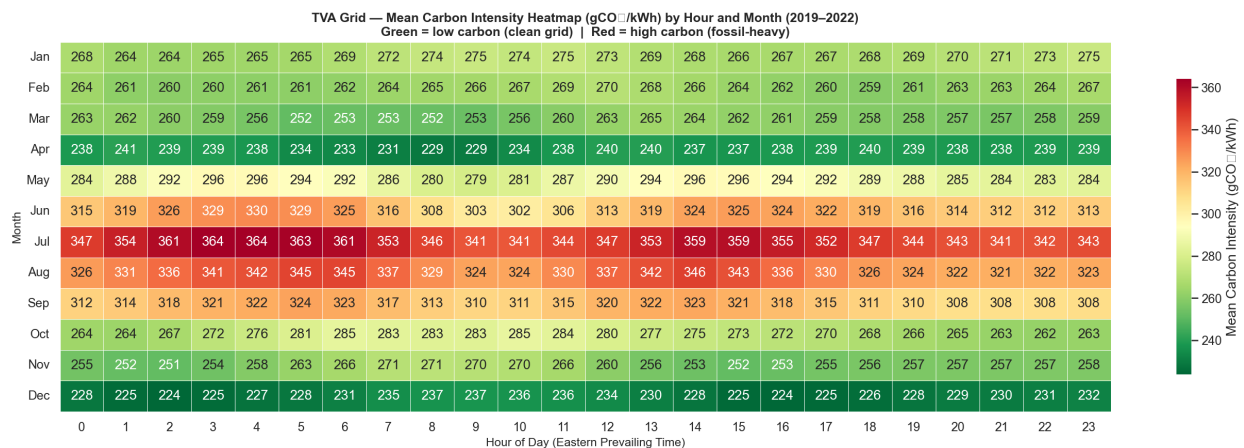
Phase 3 — Analysis

The analysis follows the three standard analytics layers — descriptive, predictive and prescriptive — each answering a different question and building on the last.

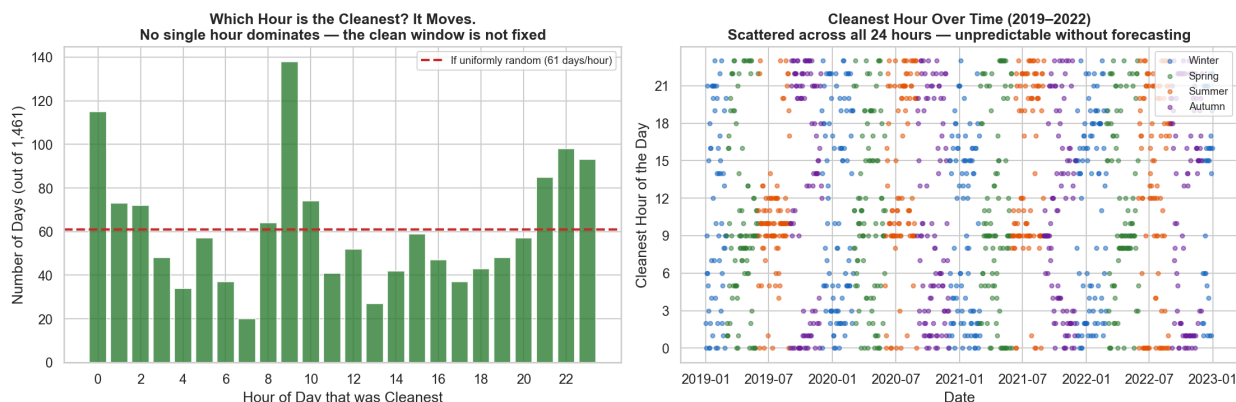
Layer	Question	In this project
Descriptive	What is happening?	When is the facility busy, and when is the grid cleanest?
Predictive	What will happen?	Grid carbon intensity over the next 48 hours
Prescriptive	What should we do?	When each flexible job should run to cut carbon

Descriptive — understanding the patterns

Using **pandas** with **matplotlib** and **seaborn**, we explored both datasets to learn when the facility draws the most power and when the grid is at its cleanest. The key discovery concerns the grid. Averaged across four years, every hour of the day looks almost equally clean — which would suggest that timing barely matters. But that average is misleading: on any individual day the gap between the cleanest and dirtiest hour is large (about 51 gCO₂/kWh), and the cleanest hour is rarely the same two days in a row, shifting unpredictably. A fixed rule such as ‘always run overnight’ would therefore miss the clean window most of the time — the only way to find it is to **forecast** it.



Grid carbon intensity by hour (columns) and month (rows) — green is clean, red is dirty. No single hour-of-day stays green all year.

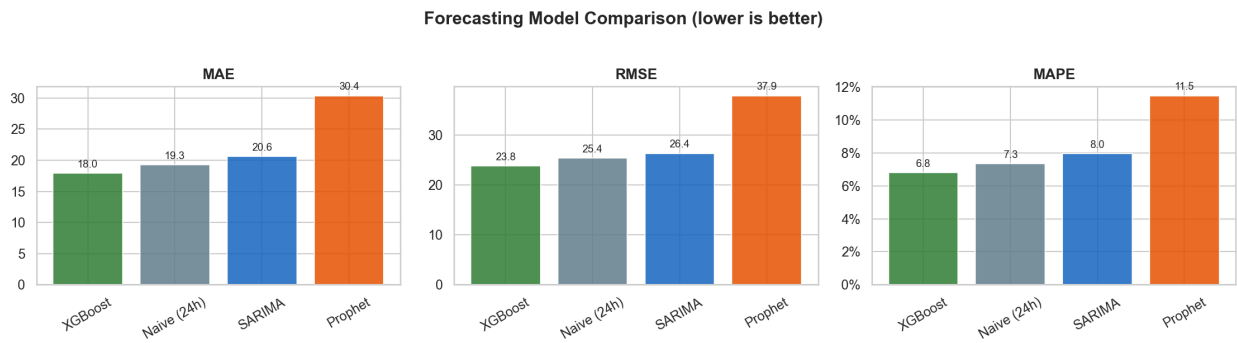


Left: how often each hour of the day was the cleanest — no hour dominates. Right: the cleanest hour plotted over time, scattered with no fixed pattern.

Predictive — forecasting carbon intensity

To know *when* the grid will be clean, we forecast its carbon intensity 48 hours ahead, benchmarking four models against a naive baseline on a held-out four-month test set using MAE, RMSE and MAPE. **XGBoost**

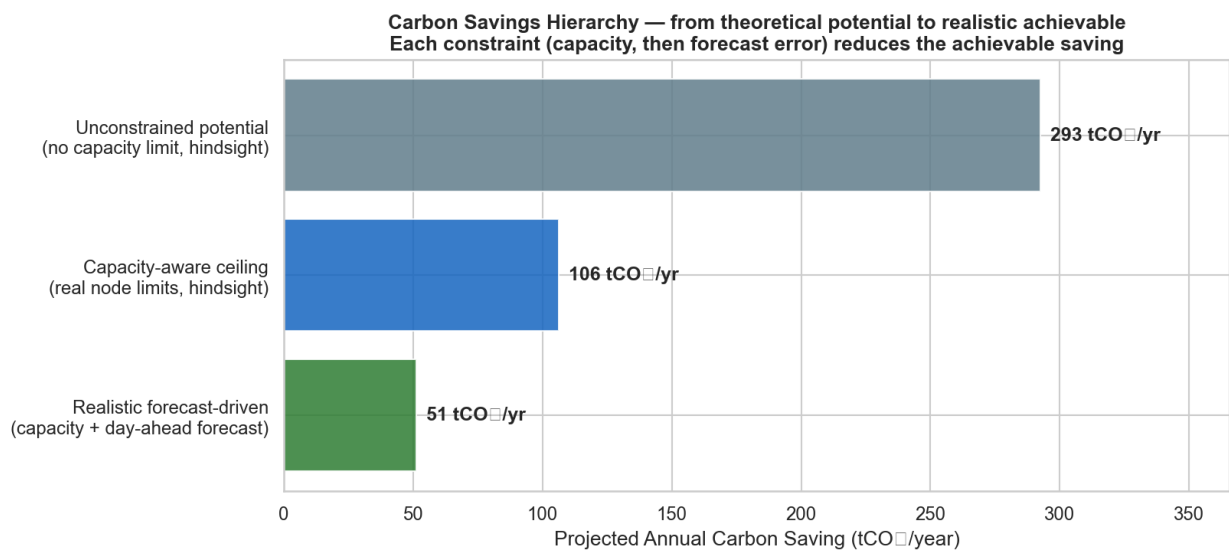
(with statsmodels, Prophet and scikit-learn as comparators) was selected as the only model to beat the baseline on all three metrics.



Model comparison on held-out data — XGBoost is best on all three error metrics, so it was chosen.

Prescriptive — optimising the schedule

The forecast is turned into scheduling decisions using a greedy heuristic (which powers the live dashboard) and a Linear Programme (the optimal benchmark), both built with **PuLP** and the open-source **CBC** solver under node-capacity and deadline limits. This layer also keeps the savings honest: once real cluster capacity and day-ahead forecast error are taken into account, the achievable saving falls well below the theoretical maximum.



Savings hierarchy — each real constraint lowers the achievable annual saving (293 → 106 → 51 tCO₂/yr).

Options considered and chosen approach

Layer	Options considered	Chosen approach & why
Descriptive	BI dashboards vs code-based EDA	Code-based EDA (pandas, seaborn) — reproducible and shares the Python pipeline
Predictive	Naive, SARIMA, Prophet, XGBoost, LSTM	XGBoost — best accuracy on held-out data; others retained as honest benchmarks
Prescriptive	Greedy, Linear Programming, stochastic optimisation	Greedy + PuLP/CBC — fast for live use plus an optimal benchmark; stochastic dr